COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	August 10, 2026	**DATE SUBMITTED:**	August 11, 2026
NAME:	Mendoza, Ryo Shin P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5.1 QUEUE

1. PROCEDURE OUTPUT

ILOA

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> r);

int main()
{

    std::queue<char> q;

    q.push('c');
    q.push('p');
    q.push('e');
    q.push('0');
    q.push('1');
    q.push('0');

    std::cout << "================================\n";
    std::cout << "Queue Implementation\n";
    std::cout << "================================\n\n";
    // Display the queue
    std::cout << "Display: ";
    display(q);
    std::cout << "================================\n\n";
```

```
// empty()
if (q.empty())
    std::cout << "Queue is empty" << std::endl;
else
    std::cout << "Queue is not empty" << std::endl;
std::cout << "================================\n\n";

// size()
std::cout << "Queue size is: " << q.size() << std::endl;
std::cout << "================================\n\n";
// front
std::cout << "Front Element: " << q.front() << std::endl;
std::cout << "================================\n\n";
// back
std::cout << "Back Element: " << q.back() << std::endl;
std::cout << "================================\n\n";
// Display
std::cout << "Queue contents:";
display(q);
std::cout << "================================\n\n";
// Push another element
q.push('X');
// Display the updated queue
std::cout << "After pushing 'X':";
display(q);
std::cout << "================================\n\n";

std::cout << "Deleting all elements....\n";
// deletion
while (!q.empty())
{
    display(q);
    q.pop();
}
```

```
    std::cout << "Final Queue:" << std::endl;
    display(q);

    return 0;
}

void display(std::queue<char> r)
{
    // Create a copy of the queue
    std::queue<char> c = r;

    while (!c.empty())
    {
        std::cout << " " << c.front();
        c.pop();
    }

    std::cout << std::endl;
}
```

Output

```
==============================
Queue Implementation
==============================

Display:  c p e 0 1 0
==============================

Queue is not empty
==============================

Queue size is: 6
==============================

Front Element: c
==============================

Back Element: 0
==============================

Queue contents: c p e 0 1 0
==============================

After pushing 'X': c p e 0 1 0 X
==============================

Deleting all elements....
 c p e 0 1 0 X
 p e 0 1 0 X
 e 0 1 0 X
 0 1 0 X
 1 0 X
 0 X
 X
Final Queue:
```

Code Analysis

This C++ program shows how a Queue data structure works using basic operations like adding, displaying, checking, and removing elements. It uses push() to add data, front() and back() to access elements, and pop() to remove items following the FIFO (First In, First Out) principle. The program also uses a display function to show the queue contents without changing the original data.

ILOB.1	
Function	**Source Code/Header File**
Creating new node and public variables	

```
template <typename T>
class qNode
{
public:
    T data;
    qNode<T> *next;
};

// Create a new node
template <typename T>
qNode<T> *new_node(T newData)
{
    qNode<T> *newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}
```

Enqueue the elements	

```
// Enqueue
template <typename T>
void enqueue(qNode<T> **frontPtr, qNode<T> **backPtr, T newData)
{
    // Create new node
    qNode<T> *newNode = new_node(newData);

    // Queue is empty
    if ((*frontPtr) == nullptr && (*backPtr) == nullptr)
    {
        *frontPtr = newNode;
        *backPtr = newNode;
        return;
    }

    // Queue is not empty
    (*backPtr)->next = newNode;
    *backPtr = newNode;
}
```

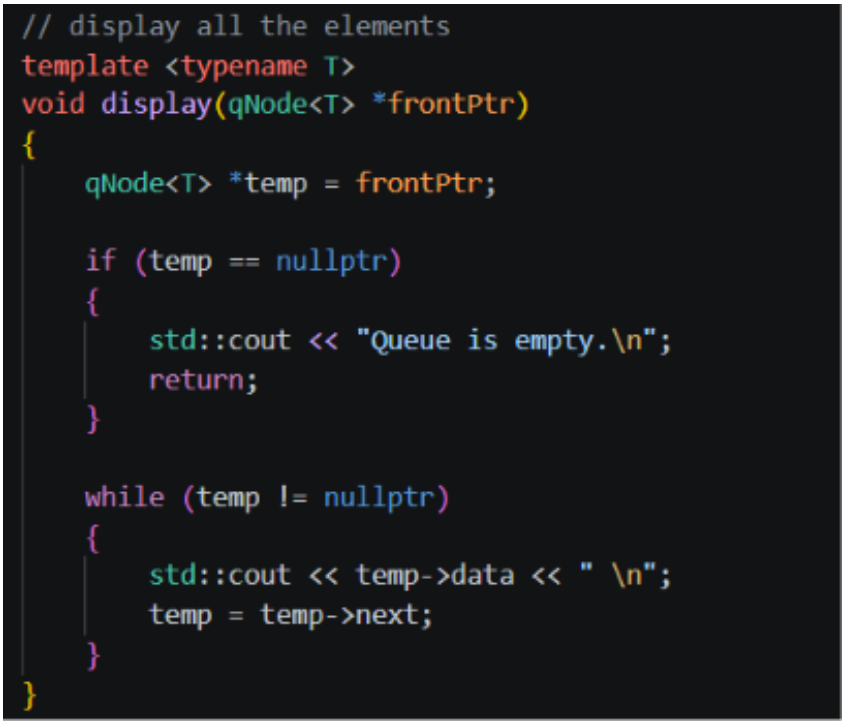
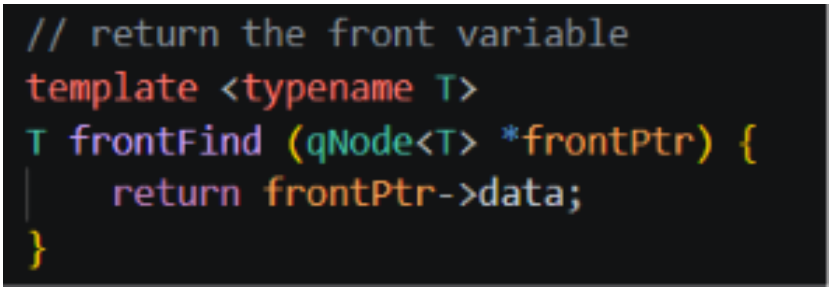
Dequeue the elements	

```
// Dequeue
template <typename T>
void dequeue(qNode<T> **frontPtr, qNode<T> **backPtr)
{
    // Queue is empty
    if (*frontPtr == nullptr)
    {
        return;
    }

    // Temporary node
    qNode<T> *deleteNode = *frontPtr;

    // Only one node
    if ((*frontPtr)->next == nullptr)
    {
        *frontPtr = nullptr;
        *backPtr = nullptr;
        delete deleteNode;
        return;
    }

    // More than one node
    *frontPtr = (*frontPtr)->next;
    delete deleteNode;
}
```

Display all the elements	
Return or print the front element	

Code Analysis

The header file contains the functions and classes needed to create and manage the queue. The qNode class defines each node that stores data and a pointer to the next node. The new_node() function creates a new node, while enqueue() adds new elements at the back of the queue. The dequeue() function removes elements from the front and updates the pointers properly. The display() function shows all elements inside the queue by going through each node, and frontFind() returns the value of the first element. Overall, the header file provides the functions needed to implement a queue using a linked list.

Main Code

```cpp
#include <iostream>
#include "queuell.h"

int main()
{
    qNode<char> *front = nullptr;
    qNode<char> *back = nullptr;

    enqueue(&front, &back, 'R');
    std::cout << "Front: " << front->data << " Back: " << back->data << std::endl;

    enqueue(&front, &back, 'Y');
    std::cout << "Front: " << front->data << " Back: " << back->data << std::endl;

    enqueue(&front, &back, 'O');
    std::cout << "Front: " << front->data << " Back: " << back->data << std::endl;

    std::cout << "\nDequeue (R)" << std::endl;
    dequeue(&front, &back);

    std::cout << "Front: " << front->data
              << " Back: " << back->data << std::endl;

    std::cout << "\nDisplay all: " << std::endl;
    display(front);

    std::cout << "Front Element: " << frontFind(front) << std::endl;

    return 0;
}
```

Output

```
Front: R Back: R
Front: R Back: Y
Front: R Back: O

Dequeue (R)
Front: Y Back: O

Display all:
Y
O
Front Element: Y
```

Code Analysis

The main file controls the execution of the queue program. It creates an empty queue using two pointers, front and back, which represent the first and last elements of the queue. The program adds elements using the enqueue() function and displays the current front and back values after each insertion. It then removes the first element using dequeue() and shows the updated queue. The display() function is used to print all remaining elements, while frontFind() is used to get the first element of the queue. Overall, the main file tests and demonstrates the basic operations of a queue.

ILOB.2

Header File

```
#ifndef QUEUEARRAY_H
#define QUEUEARRAY_H

#include <iostream>

template <typename T>
class queueArr
{
private:
    // create a pointer array
    T* q_array;

    size_t q_capacity;
    size_t q_size;

    int q_front;
    int q_back;

public:
    // prototypes to use for queueArr
    queueArr(size_t capacity);

    bool empty();
```

```
    bool full();

    size_t size();
    void Clear();

    T Front();
    T Back();

    void Enqueue(T newData);
    void Dequeue();

    ~queueArr();

    // tasks
    queueArr(const queueArr& other);
    queueArr& operator=(const queueArr& other);
};


// ============================================================
// Constructor
// ============================================================

template <typename T>
queueArr<T>::queueArr(size_t capacity)
{
    // capacity of the array
    q_capacity = capacity;

    // current size of the array
    q_size = 0;

    // initial front and back of the index
    q_front = 0;
    q_back = -1;
```

```
    // newly allocated array
    q_array = new T[q_capacity];

    std::cout << "Queue created with capacity: "
              << q_capacity << std::endl;
}


// ================================================================
// Destructor
// ================================================================

template <typename T>
queueArr<T>::~queueArr()
{
    delete[] q_array;

    std::cout << "Queue destroyed." << std::endl;
}


// ================================================================
// Empty
// ================================================================

template <typename T>
bool queueArr<T>::empty()
{
    return q_size == 0;
}


// ================================================================
// Full
// ================================================================
```

```cpp
template <typename T>
bool queueArr<T>::full()
{
    return q_size == q_capacity;
}


// ================================================================
// Size
// ================================================================

template <typename T>
size_t queueArr<T>::size()
{
    return q_size;
}


// ================================================================
// Clear
// ================================================================

template <typename T>
void queueArr<T>::Clear()
{
    q_size = 0;
    q_front = 0;
    q_back = -1;
}


// ================================================================
// Front
// ================================================================

template <typename T>
```

```cpp
T queueArr<T>::Front()
{
    // check if the queue is empty
    if (empty())
    {
        std::cout << "Queue is empty.\n";
        return T{};
    }

    // return the front element
    return q_array[q_front];
}



// ================================================================
// Back
// ================================================================

template <typename T>
T queueArr<T>::Back()
{
    // check if the queue is empty
    if (empty())
    {
        std::cout << "Queue is empty.\n";
        return T{};
    }

    // return the back element
    return q_array[q_back];
}



// ================================================================
// Enqueue
// ================================================================
```

```cpp
template <typename T>
void queueArr<T>::Enqueue(T newData)
{
    // check if the queue is full
    if (full())
    {
        std::cout << "Queue is full.\n";
        return;
    }

    // move back to the next position
    q_back = (q_back + 1) % q_capacity;

    // insert the new data
    q_array[q_back] = newData;

    // increase the size
    q_size++;
}


// ================================================================
// Dequeue
// ================================================================

template <typename T>
void queueArr<T>::Dequeue()
{
    // check if the queue is empty
    if (empty())
    {
        std::cout << "Queue is empty.\n";
        return;
    }
```

```
    // move front to the next position
    q_front = (q_front + 1) % q_capacity;

    // decrease the size
    q_size--;

    // reset indexes if queue becomes empty
    if (q_size == 0)
    {
        q_front = 0;
        q_back = -1;
    }
}


// ============================================================
// Copy Constructor
// ============================================================

template <typename T>
queueArr<T>::queueArr(const queueArr& other)
{
    // copy the capacity
    q_capacity = other.q_capacity;

    // copy the current size
    q_size = other.q_size;

    // copy front and back
    q_front = other.q_front;
    q_back = other.q_back;

    // create a new array
    q_array = new T[q_capacity];

    // copy all elements
```

```
    for (size_t i = 0; i < q_capacity; i++) {
        q_array[i] = other.q_array[i];
    }

    std::cout << "Queue copied using copy constructor."
              << std::endl;
}


// ============================================================
// Copy Assignment Operator
// ============================================================

template <typename T>
queueArr<T>& queueArr<T>::operator=(const queueArr& other)
{
    // prevent self-assignment
    if (this != &other) {
        // delete old array
        delete[] q_array;

        // copy capacity
        q_capacity = other.q_capacity;

        // copy size
        q_size = other.q_size;

        // copy front and back
        q_front = other.q_front;
        q_back = other.q_back;

        // create a new array
        q_array = new T[q_capacity];

        // copy elements
        for (size_t i = 0; i < q_capacity; i++) {
```

```
            q_array[i] = other.q_array[i];
        }
    }

    return *this;
}

#endif
```

Main Code

```
#include <iostream>
#include "queuearray.h"

int main()
{
    // Create a queue with capacity of 6
    queueArr<int> q(6);

    std::cout << "\n===== QUEUE-ARRAY TEST =====\n\n";

    // Enqueue elements
    std::cout << "Enqueue 10, 20, 30...\n";

    q.Enqueue(10);
    q.Enqueue(20);
    q.Enqueue(30);

    // Display size
    std::cout << "Queue size: " << q.size() << std::endl;

    // Display front and back
    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
```

```cpp
    // Dequeue
    std::cout << "\nDequeue...\n";
    q.Dequeue();
    std::cout << "Front after Dequeue: " << q.Front() << std::endl;
    std::cout << "Queue size: " << q.size() << std::endl;

    // Enqueue
    std::cout << "\nEnqueue 40...\n";
    q.Enqueue(40);
    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Queue size: " << q.size() << std::endl;

    // Check if empty
    std::cout << "\nIs queue empty? ";

    if (q.empty()) {
        std::cout << "Yes\n";
    }
    else {
        std::cout << "No\n";
    }

    // Check if full
    std::cout << "Is queue full? ";

    if (q.full()) {
        std::cout << "Yes\n";
    }
    else {
        std::cout << "No\n";
    }

    // Copy Constructor
    std::cout << "\n===== COPY CONSTRUCTOR =====\n";
```

```
    queueArr<int> q2(q);

    std::cout << "q2 Front: " << q2.Front() << std::endl;
    std::cout << "q2 Back: " << q2.Back() << std::endl;
    std::cout << "q2 Size: " << q2.size() << std::endl;

    // Copy Assignment Operator
    std::cout << "\n===== COPY ASSIGNMENT =====\n";
    queueArr<int> q3(6);
    q3 = q;

    std::cout << "q3 Front: " << q3.Front() << std::endl;
    std::cout << "q3 Back: " << q3.Back() << std::endl;
    std::cout << "q3 Size: " << q3.size() << std::endl;

    // Clear
    std::cout << "\n===== CLEAR =====\n";
    q.Clear();
    std::cout << "Queue size after Clear: " << q.size() << std::endl;
    std::cout << "Is queue empty? ";

    if (q.empty()) {
        std::cout << "Yes\n";
    }
    else {
        std::cout << "No\n";
    }

    std::cout << "\n===== END OF PROGRAM =====\n";

    return 0;
```

Code Analysis

The program tests a queue implemented using an array. First, it creates a queue with a capacity of 6 and adds 10, 20, and 30. It then checks the queue size, front, and back values. The Dequeue() function removes the first

element (10), making 20 the new front. The program then adds 40 to the queue. It also checks if the queue is empty or full. After that, it tests the copy constructor by copying q into q2, and the copy assignment operator by assigning q to q3. Finally, Clear() removes all elements from the original queue. The queueArr class uses a circular array, so the front and back positions can move around the array when elements are added or removed. .

Output

```
Queue created with capacity: 6

===== QUEUE-ARRAY TEST =====

Enqueue 10, 20, 30...
Queue size: 3
Front: 10
Back: 30

Dequeue...
Front after Dequeue: 20
Queue size: 2

Enqueue 40...
Front: 20
Back: 40
Queue size: 3

Is queue empty? No
Is queue full? No

===== COPY CONSTRUCTOR =====
Queue copied using copy constructor.
q2 Front: 20
q2 Back: 40
q2 Size: 3

===== COPY ASSIGNMENT =====
Queue created with capacity: 6
q3 Front: 20
q3 Back: 40
q3 Size: 3

===== CLEAR =====
Queue size after Clear: 0
Is queue empty? Yes

===== END OF PROGRAM =====
Queue destroyed.
Queue destroyed.
Queue destroyed.
```

Output Analysis

The program first displays that the queue was created with a capacity of 6. After adding 10, 20, and 30, the queue size is 3, with 10 as the front and 30 as the back. After removing 10, the front becomes 20 and the queue size becomes 2. When 40 is added, the back becomes 40 and the queue size becomes 3. The queue is

not empty and is not full. The copy constructor creates q2 with the same front (20), back (40), and size (3). The copy assignment creates q3 with the same values. When Clear() is called on q, its size becomes 0, and the program confirms that the queue is empty. At the end, the destructors display "Queue destroyed." for the queue objects.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILOC

Header File

```
#ifndef PRINTER_H
#define PRINTER_H

#include <iostream>
#include <cstring>


// ================================================================
// QUEUE ARRAY CLASS
// ================================================================

template <typename T>
class queueArr
{
private:
    T* q_array;
    int q_capacity;
    int q_size;
    int q_front;
    int q_back;

public:
    queueArr(int capacity);
    ~queueArr();
```

```
    bool empty();
    bool full();
    int size();

    T Front();

    void Enqueue(T newData);
    void Dequeue();
};


// Constructor
template <typename T>
queueArr<T>::queueArr(int capacity)
{
    q_capacity = capacity;
    q_size = 0;
    q_front = 0;
    q_back = -1;

    q_array = new T[q_capacity];
}


// Destructor
template <typename T>
queueArr<T>::~queueArr()
{
    delete[] q_array;
}


// Check if queue is empty
template <typename T>
bool queueArr<T>::empty()
{
    return q_size == 0;
```

```
}


// Check if queue is full
template <typename T>
bool queueArr<T>::full()
{
    return q_size == q_capacity;
}


// Get queue size
template <typename T>
int queueArr<T>::size()
{
    return q_size;
}


// Get front element
template <typename T>
T queueArr<T>::Front()
{
    if (empty())
    {
        return T{};
    }

    return q_array[q_front];
}


// Add element to queue
template <typename T>
void queueArr<T>::Enqueue(T newData)
{
    if (full())
```

```
    {
        std::cout << "Queue is full." << std::endl;
        return;
    }

    q_back = (q_back + 1) % q_capacity;

    q_array[q_back] = newData;

    q_size++;
}


// Remove element from queue
template <typename T>
void queueArr<T>::Dequeue()
{
    if (empty())
    {
        std::cout << "Queue is empty." << std::endl;
        return;
    }

    q_front = (q_front + 1) % q_capacity;

    q_size--;

    if (q_size == 0)
    {
        q_front = 0;
        q_back = -1;
    }
}


// ============================================================
// JOB CLASS
```

```cpp
// ================================================================

class Job
{
private:
    int id;
    char user[50];
    int pages;

public:
    // Default values allow the queue to create an empty array
    Job(int jobID = 0, const char* userName = "", int numberOfPages =
0) {
        id = jobID;

        std::strncpy(user, userName, 49);
        user[49] = '\0'; // can be store up to 49 char of letters to
the names

        pages = numberOfPages;
    }

    void display()
    {
        std::cout << "Job ID: " << id << std::endl;
        std::cout << "User: " << user << std::endl;
        std::cout << "Pages: " << pages << std::endl;
    }
};



// ================================================================
// PRINTER CLASS
// ================================================================

class Printer
{
```

```
private:
    queueArr<Job> jobs; //objects

public:
    // Constructor
    Printer(int capacity) : jobs(capacity)
    {
    }

    // Add a new print job
void AddJob(Job newJob)
    {
        if (jobs.full())
        {
            std::cout << "Printer queue is full." << std::endl;
            return;
        }

        jobs.Enqueue(newJob);
        std::cout << "Job added to printer." << std::endl;
    }

    // Process one job
void ProcessJob()
    {
        if (jobs.empty())
        {
            std::cout << "No jobs to print."
                      << std::endl;
            return;
        }

        Job currentJob = jobs.Front();

        std::cout << "\nPrinting job..." << std::endl;

        currentJob.display();
```

```
        std::cout << "Job completed."
                  << std::endl;

        jobs.Dequeue();
    }

    // Process all jobs
void ProcessAll()
    {
        while (!jobs.empty())
        {
            ProcessJob();
        }
    }
};

#endif
```

Main Code

```
#include <iostream>
#include "printer.h"

int main()
{
    Printer printer(10); // storing 10 jobs

    Job job1(101, "Ryo", 8);
    Job job2(102, "Khaira", 10);
    Job job3(103, "Jabez", 6);
    Job job4(104, "Ivan", 7);
    Job job5(105, "Xyle", 9);

    std::cout << "===== SHARED PRINTER SIMULATION =====" << std::endl;

    std::cout << std::endl;
```

```
    std::cout << "Adding jobs..." << std::endl;

    printer.AddJob(job1);
    printer.AddJob(job2);
    printer.AddJob(job3);
    printer.AddJob(job4);
    printer.AddJob(job5);

    std::cout << std::endl;
    std::cout << "===== PROCESSING JOBS =====" << std::endl;

    printer.ProcessAll();

    std::cout << std::endl;
    std::cout << "===== ALL JOBS COMPLETED =====" << std::endl;

    return 0;
}
```

Code Analysis

The program uses an array-based circular queue to simulate a shared printer. The Job class stores the job ID, user name, and number of pages, while the queueArr class manages the jobs using Enqueue(), Dequeue(), and Front(). The Printer class adds jobs to the queue and processes them one at a time using the FIFO (First In, First Out) principle. The printer has a capacity of 10 jobs, while 5 jobs are added in the simulation.

Output

```
Adding jobs...
Job added to printer.
Job added to printer.
Job added to printer.
Job added to printer.
Job added to printer.

===== PROCESSING JOBS =====

Printing job...
Job ID: 101
User: Ryo
Pages: 8
Job completed.

Printing job...
Job ID: 102
User: Khaira
Pages: 10
Job completed.

Printing job...
Job ID: 103
User: Jabez
Pages: 6
Job completed.

Printing job...
Job ID: 104
User: Ivan
Pages: 7
Job completed.

Printing job...
Job ID: 105
User: Xyle
Pages: 9
Job completed.

===== ALL JOBS COMPLETED =====
```

Output Analysis

The output shows that the five jobs were successfully added to the printer queue. The printer processes the jobs in the same order they were submitted: Ryo (Job 101), Khaira (Job 102), Jabez (Job 103), Ivan (Job 104), and

Xyle (Job 105). Each job displays its ID, user, and number of pages before being marked as completed. This confirms that the queue correctly follows the FIFO principle, where the first submitted job is printed first. .

Conclusion

Overall, I learned how a queue works using the FIFO (First In, First Out) principle and how to implement it using a linked list. I learned how to create classes, add jobs to the queue, and process them in the correct order. I also learned how front, rear, nodes, and pointers work together to manage the queue. This activity helped me understand how queues can be used in real-life situations, such as managing print jobs in a shared printer.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, QuillBot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Assistance, Grammar Correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I confirm that Ai did not replace my critical thinking.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.